

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED:-

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

Annexure A

Scenario Based Assignment

Code of Conduct:

I _____ BAIRAGANI DINESH -192372189 _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:BAIRAGANI DINESH

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a

flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic

estimate (straight-line distance), Partial real-time updates about blocked paths and Variable

movement costs depending on terrain and weather. However, due to urgency, the drone must

quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety.

ANS:

1. Scenario Analysis: Emergency Drone Delivery

i. Greedy Best-First Search Behavior, Strengths & Weaknesses

- **Behavior:** Chooses nodes purely based on the heuristic function $h(n)$ (straight-line distance to the target). It expands nodes that appear closest to the destination without considering accumulated travel costs.
- **Strengths:**
 - Extremely fast execution with low memory overhead, providing quick decision-making under high urgency.
- **Weaknesses:**
 - **Ignores actual path costs:** May lead the drone into high-risk weather zones or severe terrain penalties.
 - **Incomplete under uncertainty:** Can get stuck in dead ends (e.g., newly blocked paths/flooded areas) and requires costly backtracking.

ii. A* Search Behavior & Cost Balancing

- **Behavior:** Evaluates paths using $f(n) = g(n) + h(n)$, where:
 - $g(n)$ is the actual path cost from start to current location (terrain difficulty, weather factors, risk penalties).

- $h(n)$ is the estimated straight-line distance to the target.
- **Balancing:** $g(n)$ prevents taking dangerously high-cost or high-risk paths, while $h(n)$ directs the search towards the goal. This dual approach ensures the drone finds the shortest, safest route while maintaining directional efficiency.

iii. Impact of Heuristic Accuracy

- **Admissible & Consistent Heuristic:** Straight-line distance never overestimates actual distance ($h(n) \leq h^*(n)$).
- **Impact on A*:** Guarantees an optimal (shortest/safest) route. Higher heuristic accuracy prunes unnecessary search paths and speeds up computation.
- **Impact on Greedy Best-First Search:** Accuracy helps guide the drone in a straight line, but if severe terrain or blocked paths lie along that direct path, an accurate $h(n)$ can mislead the algorithm into dead ends.

iv. Impact of Dynamic Changes (Blocked Paths)

- **Search Performance Degraded:** Static offline plans become invalid when real-time updates report blocked routes or severe weather changes.
- **Re-planning Overhead:** Both standard algorithms must restart or heavily recalculate paths from scratch upon encountering obstacle updates, leading to delays and high memory usage.

v. Final Recommendation & Justification

Recommended Approach: Dynamic A* (such as D* Lite) or Real-Time A*

- **Optimality & Safety:** Uses $f(n) = g(n) + h(n)$ to account for actual weather/terrain costs and risk penalties, ensuring optimal and safe flight paths.
- **Real-Time Efficiency:** Unlike basic static A*, variants like D* Lite re-plan routes locally upon receiving partial updates (blocked paths) without recomputing the entire search space, making it ideal for dynamic, emergency missions.

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across

hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption

and Traffic congestion. The system must continuously adjust signals based on traffic flow.

However the search space is extremely large, traffic patterns change dynamically and the

system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability.

ANS:-

Scenario Analysis: Smart City Traffic Signal Optimization

Problem Formulation & Traditional Search Inefficiency

- **Optimization Formulation:**
 - **State Space:** All possible combinations of signal timing parameters across hundreds of intersections (e.g., green light durations, offsets, phase sequences).
 - **Objective Function:** Minimize cost $C = w_1(\text{Waiting Time}) + w_2(\text{Fuel Consumption}) + w_3(\text{Congestion Index})$.
- **Why Traditional Search Fails:** Uninformed (e.g., BFS/DFS) and exhaustive searches fail because the search space is combinatorial ($O(b^d)$) across hundreds of nodes. Evaluating every traffic state in real time is computationally impossible.

i. Hill Climbing Behavior & Limitations

- **How It Works:** Starts with a baseline signal timing configuration and iteratively modifies neighboring signal timings, moving only to configurations that yield a lower cost function.
- **Limitations:**

- **Greedy Nature:** Only accepts immediate improvements, making it unable to explore non-obvious configurations that temporarily increase local delay to achieve global flow.
- **Memoryless:** Does not retain information about previously visited states.

ii. Real-World Issues: Local Maxima, Plateaus, & Ridges

- **Local Maxima (Suboptimal Traps):** A signal configuration minimizes waiting time for a small neighborhood of intersections, but causes severe bottlenecking two streets over. Hill Climbing gets stuck here because any single-step change worsens immediate traffic.
- **Plateaus (Flat Areas):** Changing green light times by a few seconds across multiple intersections produces zero noticeable change in overall congestion. Hill Climbing makes random or zero-progress moves, slowing down real-time responsiveness.
- **Ridges (Narrow Paths):** Coordinated traffic waves ("green waves") require simultaneous adjustments across multiple contiguous intersections. Hill Climbing can only alter one variable at a time, making it unable to navigate the ridge effectively.

iii. Overcoming Limitations with Advanced Heuristics

- **Simulated Annealing (SA):**
 - **Mechanism:** Allows probabilistic "bad moves" (worsening signal times) early on (high temperature T) to jump out of local optima.
 - **Cooling Schedule:** As time progresses (T decreases), it gradually shifts to pure exploitation (Hill Climbing) around a global minimum.
- **Genetic Algorithms (GA):**
 - **Mechanism:** Maintains a population of signal schedules (chromosomes).
 - **Crossover & Mutation:** Combines successful signal timings from different grid sectors (crossover) and introduces random adjustments (mutation) to ensure diversity and escape plateaus/ridges.

iv. Recommendation & Justification

Recommended Approach: Genetic Algorithms (GA) *(or a Hybrid GA + Local Search approach)*

- **Scalability:** Handles vast, high-dimensional search spaces across hundreds of intersections by evaluating candidate signal plans in parallel populations rather than a single trajectory.
- **Adaptability:** Crossover and continuous mutation allow the algorithm to dynamically adapt to sudden traffic spikes (e.g., accidents or rush-hour shifts) without getting trapped in outdated signal patterns.

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It

must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited

sensing capability and make decisions without a complete map. The rover updates its

knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- Explain why this problem requires an online search agent instead of offline search**
- Analyze the challenges of partial observability and dynamic environments**
- Describe how the rover builds and updates its internal model**
- Compare online search with uninformed and informed search strategies**
- Justify the most suitable approach considering uncertainty, adaptability, and safety?**

ANS:-

Here is a concise, structured breakdown for **Question 3:**

3. Scenario Analysis: Autonomous Mars Rover Exploration

i. Online Search vs. Offline Search Requirement

- **Offline Search Inefficiency:** Offline search requires a complete, static map beforehand to compute an entire plan from start to finish before moving. On Mars, the environment is unknown and unmapped.

- **Why Online Search is Essential:** The rover must intertwine **computation and action**—exploring, sensing its immediate surroundings, updating its beliefs, and making local step-by-step decisions in real time.

ii. Challenges of Partial Observability & Dynamic Environments

- **Partial Observability:** Sensors (cameras, LiDAR) have limited range and line-of-sight. Blind spots behind rocks or craters prevent the rover from knowing what lies ahead until it gets close.
- **Dynamic Environment & Uncertainty:** Terrain conditions shift (e.g., loose sand, wheel slip, dust storms blocking vision). An action calculated to move 1 meter forward might only result in 0.6 meters of actual movement due to slippage.

iii. Building & Updating the Internal Model

- **Perception & Mapping:** Onboard sensors gather point-cloud data to create a local **Occupancy Grid Map** (classifying space as *free*, *traversable*, *hazard*, or *unexplored*).
- **State Estimation (Localization):** Uses sensors (IMU, visual odometry) to estimate its current pose (x, y, θ) relative to mapped features.
- **Incremental Updates (SLAM):** Integrates newly sensed terrain into its global belief map, re-evaluating risk costs and path feasibility continuously as it moves.

iv. Comparison: Online Search vs. Uninformed & Informed Strategies

Feature	Uninformed Search (e.g., BFS/DFS)	Informed Search (e.g., Static A*)	Online Search (e.g., Learning Real-Time A*)
Map Requirement	Complete prior map required.	Complete prior map required.	Operates in partially/fully unknown maps.
Execution Style	Compute complete path → Execute.	Compute optimal path → Execute.	Interleaves planning, execution, and sensing.

Feature	Uninformed Search (e.g., BFS/DFS)	Informed Search (e.g., Static A*)	Online Search (e.g., Learning Real-Time A*)

v. Final Recommendation & Justification

Recommended Approach: Real-Time / Online Replanning Search (e.g., SD^* Lite or Learning Real-Time A^* - LRTA*)*

- **Safety:** Prevents catastrophic collisions by evaluating sensor data before every move, ensuring the rover never steps into unverified or hazardous ground.
- **Adaptability:** Continuously updates heuristic values $h(s)$ based on discovered terrain obstacles, avoiding the massive computational cost of re-calculating global paths from scratch.
- **Uncertainty Management:** Gracefully handles partial observability by maintaining a probabilistic belief map while making localized, robust movement decisions.

4. Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping

exams, Limited number of exam halls, Certain subjects must be scheduled before others,

Faculty availability constraints, Special accommodations for some students, The complexity

increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability ?

ANS:-

Here is a structured, medium-length breakdown for **Question 4**:

4. Scenario Analysis: Exam Timetable Generation (CSP)

CSP Formulation: Variables, Domains, & Constraints

1. Variables (V)

- $C = \{c_1, c_2, \dots, c_n\}$: The set of all courses/exams that need to be scheduled.

2. Domains (D)

- For each course c_i , its domain consists of possible tuples of (Timeslot, Hall, Invigilator/Faculty):

$$D_i = \left\{ \left(\text{slot}_k, \text{hall}_m, \text{faculty}_p \right) \right\}$$

3. Constraints (C)

- **Hard Constraints (Must be strictly satisfied):**
 - **No Student Overlap:** If two courses share enrolled students, $\text{slot}(c_i) \neq \text{slot}(c_j)$.
 - **Hall Capacity & Availability:** Total students taking exams in hall $_m$ at slot $_k$ must not exceed the hall's capacity. Two exams cannot occupy the same hall simultaneously unless capacity allows splitting.
 - **Precedence Constraints:** Prerequisites or designated ordering (e.g., c_i must occur before c_j).
 - **Faculty Availability:** faculty $_p$ cannot invigilate two exams at the same timeslot.
- **Soft Constraints (Optimization preferences):**
 - Special accommodations (e.g., placing specific exams in accessible ground-floor halls).
 - Avoiding back-to-back exams for students on the same day.

ii. Backtracking Search Mechanics

- **How It Works:** Backtracking is a depth-first search (DFS) tailored for CSPs. It assigns values to variables one by one:
 1. Pick an unassigned course c_i .
 2. Assign a candidate tuple $(\text{slot}_k, \text{hall}_m, \text{faculty}_p)$ from D_i .
 3. Check if this assignment violates any hard constraints with previously assigned courses.
 4. **If valid:** Move to the next course.
 5. **If a constraint is violated (Dead-End):** Backtrack to the previous course assignment and try a different domain value.

iii. Improving Efficiency with Constraint Propagation

Standard backtracking evaluates constraints *after* making an assignment. Constraint propagation prunes invalid values *before* exploring them:

- **Forward Checking:** Whenever course c_i is assigned a timeslot, the algorithm immediately looks ahead at all unassigned neighbor courses c_j (e.g., courses sharing students or halls) and removes conflicting options from their domains (D_j). If any D_j becomes empty, the algorithm backtracks immediately without wasting search steps.
- **Arc Consistency (AC-3 Algorithm):** Maintains consistency across pairs of variables. If a value in D_i has no valid corresponding value in D_j satisfying the constraint, that value is permanently deleted from D_i , drastically shrinking the overall search space.

iv. Real-World Challenges & Scalable Strategy

Real-World Challenges

- **Combinatorial Explosion:** As the number of courses and students grows, the search space expands exponentially.
- **Dynamic Changes:** Late changes in faculty availability or room allocations can invalidate a complete solution.

Recommended Scalability Strategy

Hybrid Approach: Min-Conflicts Local Search + Constraint Backtracking with MRV Heuristic

- **Variable/Value Heuristics:** Use **Minimum Remaining Values (MRV)** to pick the most constrained course first, and **Least Constraining Value (LCV)** to assign options that preserve maximum flexibility for remaining courses.

- **Local Search (Min-Conflicts):** For very large universities, start with a complete (possibly invalid) schedule and iteratively reassign variables to minimize total constraint violations. This scales far better than exhaustive DFS for ultra-large instances.

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta

Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The

AI must compete against human players, Make decisions within strict time limits, Evaluate

complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- Explain how the Minimax algorithm models this problem
- Analyze the computational challenges of large game trees
- Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- Design an evaluation function and justify the factors considered
- Recommend a strategy for balancing optimality and real-time performance ?

ANS:-

5. Scenario Analysis: Strategic Game AI (Minimax & Alpha-Beta Pruning)

i. How Minimax Models the Problem

- **Game Tree Representation:** The game is modeled as a zero-sum, two-player game tree where nodes represent board/game states and branches represent valid actions.
- **Roles:**

- **MAX (AI Player):** Seeks to maximize the evaluation score to choose the best strategic outcome.
- **MIN (Opponent/Human):** Assumed to play rationally to minimize MAX's score.
- **Decision Rule:** Minimax works recursively bottom-up from leaf nodes, selecting the maximum score when it is MAX's turn and the minimum score when it is MIN's turn.

ii. Computational Challenges of Large Game Trees

- **Exponential Branching Factor (b^d):** Real-time strategy (RTS) games have massive branching factors b (hundreds of possible actions per move) and large search depths d .
- **Time Overhead:** Traversing the full game tree to terminal states is impossible within real-time constraints (often milliseconds per move).
- **Memory Limitations:** Storing or traversing deep recursion trees in memory slows down processing and risks system timeouts.

iii. Efficiency Improvement via Alpha-Beta Pruning

Alpha-Beta pruning reduces the number of evaluated nodes without altering the final outcome by tracking two parameters:

- α : The best score MAX is guaranteed so far.
- β : The best score MIN is guaranteed so far.

Pruning Logic:

- **β -Cutoff:** If MIN discovers a branch whose score is less than or equal to α (score $\leq \alpha$), MAX will never allow the game to reach that state. The remaining sub-branches under that node are pruned.
- **α -Cutoff:** If MAX discovers a branch whose score is greater than or equal to β (score $\geq \beta$), MIN will avoid it.
- **Efficiency:** In an optimal move-ordering scenario, time complexity reduces from $\mathcal{O}(b^d)$ to $\mathcal{O}(b^{d/2})$, effectively doubling the searchable depth in the same amount of time.

iv. Evaluation Function Design & Factor Justification

$$f(s) = w_1 \cdot \text{Resource Count} + w_2 \cdot \text{Unit Advantage} + w_3 \cdot \text{Territorial Control} - w_4 \cdot \text{Base Damage}$$

- **Factors & Justification:**

- **Resource & Economy Count (w_1):** Measures long-term sustainability and ability to produce units.
- **Military Strength / Unit Advantage (w_2):** Quantifies immediate tactical combat power.
- **Territorial Control (w_3):** Captures strategic positioning and map vision.
- **Base Health / Risk Penalty (w_4):** Heavily penalizes vulnerability to prevent direct defeat.

v. Recommended Strategy: Real-Time vs. Optimality Balance

Recommended Approach: Iterative Deepening Minimax with Alpha-Beta Pruning, Transposition Tables, & Move Ordering

- **Depth-Limited Search:** Replaces terminal depth with a fixed search depth limit d , applying the evaluation function at non-terminal leaf nodes.
- **Iterative Deepening:** Runs search at depth 1,2,3, ... and halts immediately when the move timer expires, returning the best complete depth result.
- **Move Ordering & Transposition Tables:** Uses dynamic move ordering (evaluating strong moves first) to maximize pruning efficiency, combined with transposition tables (memoization) to avoid re-evaluating duplicate game states.